ALMA MATER STUDIORUM – UNIVERSITÀ DI BOLOGNA

Corso di Laurea Magistrale in Digital Transformation Management

Fancy Title

Tesi di laurea in:
Business Intelligence

Relatore
Matteo Golfarelli

Candidato
Andrea Giovanardi

II Sessione di Laurea
Anno Accademico 2025-2026

Abstract

Max 2000 characters, strict.

Optional. Max a few lines.

Contents

List of Figures

List of Listings

Chapter 1

Introduction

1.1 Context

During one of the audited weeks of my internship, the operational dashboard flagged a clear productivity gap at the station. The AI summary built on top of it explained that difference with full confidence, but the explanation produced was factually incorrect. It pointed to a structural metric that had little to do with the real cause of the difference, proposed corrective actions and root analyses that no manager could realistically carry out. I noticed the error, that morning, because I was familiar with the station and the metric, but a manager reading the same paragraph on a busy day, or less experienced, could have easily taken that for true and acted on it.

That episode marked my thesis beginning and frames the general problem it addresses: what happens when a Large Language Model (LLM) is placed on top of a Business Intelligence (BI) dashboard and asked to turn numbers into decisions? That's a key question considering the setting of a Delivery Station (DS). It is the final node of the logistic network, the place where packages are sorted in the early morning and loaded onto routes to be delivered to final customers. The station where I worked, DFV2 in Udine, runs on a standard daily rhythm: plan the day, run it, measure what happen, adjust for tomorrow. The manager controls the rhythm through a large set of indicators that are read every day to determine if

any correction is needed. Normally the window to act is narrow so it is important that a manager doesn't get the picture late or even wrong otherwise the chances to fix it would be lost.

Among all the indicators that we used at the station, one metric acts as the headline figure, a single number that capture how efficiently the station operates. Most of the days this number is close to the planned one, but not exactly on it and that gap is what a manager has to explore, understand and explain. A process that is anything but trivial. This difference is the combined effect of many separate factors, some of them are internal to the station while the rest of them are completely external, and working out which factor and its weight is a task that, before this project, was done completely by hand.

When a station misses its target, the designated manager has to produce a written justification, internally referred to as the "bridge", based on a weekly Excel report that states how the previous one had closed. A manual and slow activity that said nothing about how the current week was going while there was still time to act on it.

This is the exact scenario where a LLM could be the right fit. A model capable of reading the numbers and turning them into paragraphs within seconds, replacing hours of manual assembly. The downside is that the same flexibility that makes it so powerful and useful is also the source of risks. A language model hallucinates on numbers, quietly recomputing a value or inventing one that looks plausible, and it explains causes with the same fluency whether the cause is real or not. For a tool that is used to feed decisions on the floor those errors are not acceptable, and they must be reduced as much as possible, while keeping in mind that hallucinations cannot be avoided at 100%. A manager must be able to safely rely on the output of the model while, at the same time having a clear view of the related data.

1.2 Criticalities

The main difficulty of this project is the grounding problem, intended in a stricter sense than the one commonly used in the literature. It can be decomposed into two

main requirements that need to hold together. The first is numerical: the number must correspond exactly to the value behind the dashboard, not an approximation of it. The second one is causal: every explanation and every recommendation must come from a well-defined set of causes that the domain admits, not from everything that seems reasonable. Figure 1.1 shows the two requirements and the related failure points.

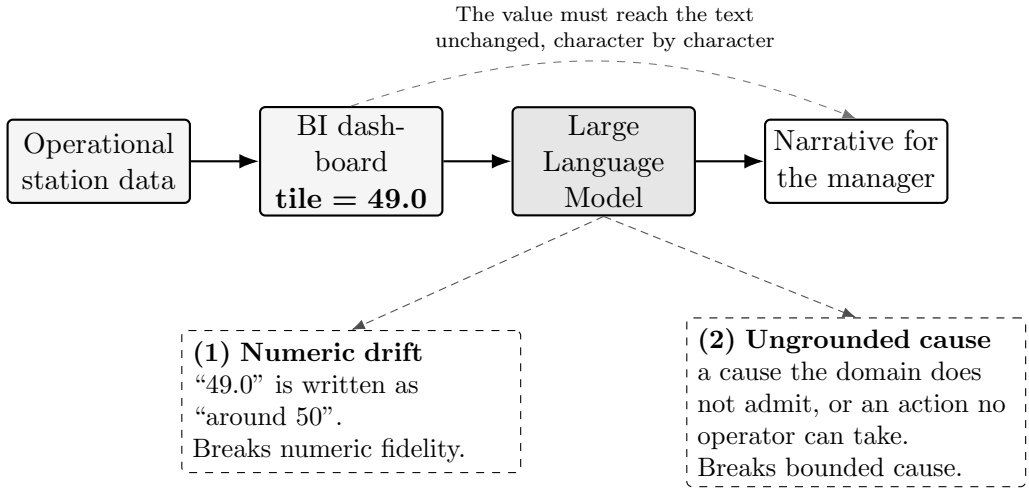

Figure 1.1: From dashboard value to narrative, and the two points where grounding breaks: a number that drift (1) and a cause the domain does not admit (2).

Consider the numerical side first. The one that is shown to the user must be equal to the retrieved one, if that value is close but not exact a decision taken on the base of it can still be wrong. This is more demanding than the usual faithfulness a retrieval system offers, where a claim counts as grounded if it is broadly supported by some documentation. For Operational BI a broad support is not enough. The number that is shown to the final user and the one that is retrieved by the code must be the same, otherwise any decision that is taken solely based on the Artificial Intelligence (AI) generated text, when a figure is close but not exact, can potentially be wrong.

The causal side is even harder. Reporting a number without changing its value is a matter of staying true to the base data. Explaining why we have that precise value, instead, is connected to the domain knowledge. The set of plausible-sounding causes for any gap is enormous, while the possible set of causes admitted by the

domain is small and specific. A narrative defined as "useful" has to stay inside this small set. It should behave like an expert analyst that among the possible explanations of the gap, chooses only the feasible ones and consequently actions that a manager can adopt in order to improve the situation. Domain constraint without numerical fidelity buys very little, a right explanation based on a wrong input is meaningless.

Those two requirements matter even more because a wrong output does not stay only on the screen. A manager uses it as the base for decisions that influence the whole productivity. The standard here is therefore demanding, above all on the numbers: a number must be right every time, because one confident mistake, acted upon, can undo a week of planning. The causal side is a different story: the most I can require is that the explanation stays inside what the domain admits. In particular, this last part is where most of the failure I observed were located.

1.3 Framework

The two criticalities highlighted in the previous section shape the questions that guide the thesis. I framed the work around one main question and two subsequent ones, each at a different level.

The first question is empirical and comes before thinking about any solution: what are the common patterns of grounding failure that occur when a language model generates a narrative over a high-density BI dashboard? I wanted a clear picture of how the model fails derived from the field directly, rather than borrowed from the literature. This decision was motivated by the intention to build a tailored and fix design around the real failures.

From the answer to this primary question two design questions follow. The second asks what architecture can be implemented to constrain the faithfulness of the model towards both the numerical and causal sides at once, rather than patching each slip in isolation? The last question is narrower: what role can a curated knowledge tree, a structured map of causes and feasible actions, play as the base structure that bounds what the model is allowed to say?

Those three questions mark the trace that next chapters will follow. The primary is answered by a catalogue of failure modes derived from the internship itself, the second and third by the framework that has been built to mitigate those errors and tested against the same catalogue.

Structure of the Thesis The remainder of this document is organized as follows:

- **Chapter 2** reviews the state of the art and related works, exploring current paradigms for LLM integration, evaluating hallucination metrics, and positioning the work with respect to current methodologies.
- **Chapter 3** introduces the case study, detailing the operational setting, the core metric, the pre-AI baseline, and the single-case methodology adopted during the internship.
- **Chapter 4** presents the proposed solution, detailing the conceptual innovation of the three-contract framework (tile-as-source, curated knowledge tree, and sanitizer) and the system architecture.
- **Chapter 5** focuses on the results and evaluation, analyzing the catalog of grounding failures and providing both a quantitative evaluation on real operational data and real-world qualitative evidence.
- **Chapter 6** discusses the findings, highlighting the thesis contributions and generalizability, acknowledging limitations, and outlining future research directions.

Chapter 2

State of the Art and Related Works

Chapter 1 framed the problem as two requirements that have to hold together. Every number that appears in the generated text must correspond to the actual value of it and every explanation must come from the well-defined domain. This chapter focus on the existing literature thought that double lens. For each family of methods, I run the same two checks: does it guarantee the truthiness of the number, and does it bound the cause? The map this build is what the last section uses to present the work.

2.1 The grounding problem

When we talk about grounding, in terms of natural language processing, usually means that the output is supported by the source. Retrieval-Augmented Generation (RAG) made this idea popular: a claim counts as grounded when retrieved evidence backs it [1]. There are two main failure modes as stated by the survey on hallucination in text generation [2, 3]. Intrinsic hallucination is text that contradicts the source, and extrinsic hallucinations is text that adds something the source cannot confirm. The same literature go further by separating faithfulness as being coherent with the input given, from factuality intended as the truth in

the world. A text can be faithful but not factual if the input is wrong, but the narrative report it coherently. At the same time, a text can be factual but not faithful if it states something right in the world, but not coherent with the input. The requirement that matter for BI is only the first one.

Operational BI requires a higher standard. The generic support is not enough. A figure of 49.0 is either reproduced exactly or it is wrong, any representation that deviates from the defined one is non-correct. As anticipated in the introduction: a narrative is grounded when every quantitative token is traceable, character by character, to a structured field, and every comparative statement agrees in sign with it. This aligns more with faithfulness rather than factuality, and it is tighter than both because it asks for fidelity to an exact value and not for consistency to with a passage. Controlled table-to-text work, the ToTTo dataset for instance [4], push the generation in that direction, by asking the model to describe only the given cells without any deviation.

The casual half has no real equivalent in the standard taxonomy. A phrase can have all the right numbers, but still name a cause that does not exist or an impossible solution. The standard question when talking about hallucinations is the following: is the claim supported? Rather than: does this causal belong to the domain defined set? The second question is the dividing line between BI and open-domain generation, and in fact is the reason why a single definition of grounding is not enough.

2.2 Retrieval-Augmented Generation (RAG)

RAG conditions the model by retrieving passages pulled from a corpus of documents, and concatenates them as context into the model's input [1]. It cuts extrinsic hallucinations, since now, the model has something to lean on without inventing. It has become the standard way to keep a generator close to a body of documents. The evaluation framework follows the same logic: RAGAS scores the faithfulness, defined as the degree to which a claim is entailed in the context retrieved [5]. Does the generated phrase "followed" from the passages? If yes, it

is considered grounded.

Here the unit of grounding is the passage, and the test is the semantic support. For a textual response this is reasonable, but not for a number. If a passage mention a figure "in the region of fifty" entails a sentence that says "around fifty", but that sentence fails the operational test I set above. Retrieval works by similarity, not by schema identity. RAG has no concept of canonical identity, so it cannot distinguish between an authoritative value and one that resemble it. RAG is necessary, but not sufficient, since it has no guarantee on the value.

2.3 Prompt engineering and reasoning

Lots of research focus on the prompt to improve the output, shifting the focus on how something is asked rather than the model itself. Chain-of-thought prompting asks the model to reason by decomposing the work in steps[6]. Self-consistency creates many reasoning paths and keeps the most voted one [7]. Instruction tuning with human feedback (RLHF) aligns the model to what the user really want by moving the work towards what is really useful [8]. All those methods lift the average quality of what comes out, not the single case.

None of them enforce something. They simply make the correct answer more likely, but they do not make the wrong one impossible. As reported later, even capable model with a cured but generic prompt tend to invent numbers. Good instructions alone cannot close this gap, but why does it lie to us? When sending a message to any LLM we are making a request not a command, and being the model itself a probabilistic generator, it is free to follow our directions or not. On the casual side, prompt engineering helps, it makes the model reason tidier, and falling on fewer off-topic causes. On the operational side it cannot supply the hard guarantee that the operation use needs, since a well done prompt engineering only push the odd side, no amount of it can make the value certain.

2.4 Tool use and program-aided generation

A different family that gives the model access to external tools. Reasoning + Acting (ReAct) represents an iterative cycle composed of thought, action and observation. It introduces the possibility to perform a search call [9]. Toolformer is a similar approach but capable of deciding on its own when to call an Application Programming Interface (API) [10]. Program-Aided Language (PAL) models performs a step forward by generating the code that automatically perform an operation, for example, instead of guessing it [11]. Consequently, the model skips the hallucination risk associated with logic. In particular this last idea is close to the one I adopted: taking all the numbers out of the model hands.

All those methods share one limit. They do a great job in securing the data, meaning the input likely will be correct, but they let the model move on this information without any control or boundary. Fetching the right value is only half of the job, the other half involves representing it correctly. Nothing with these techniques checks that the number that comes out in the sentence, is the same that went into the prompt. The tool fixes the input. The output is still free, and a free output can still drift.

2.5 Constrained and guided generation

When we talked about constrained decoding we refer to a family of methods that force the output into a defined shape independently of the content. Language Model Query Language (LMQL) treat the model not as text but as a small program characterized by a set of constrains [12]. A typical obligation can be: the response must be shorter than twenty token, and the system code will enforce that. Parsing Incrementally for Constrained Auto-Regressive Decoding (PICARD) bring this idea to Structured Query Language (SQL). It is an approach that by parsing incrementally, and so at each generation step, reject any token that would make the query invalid, based on the applied constraints. The final result is a query that is syntactically valid, otherwise any token beaking the syntax is rejected and regenerated [13].

Constrained decoding is the closest approach to the sanitizer I implemented, and described in chapter 4. This is motivated by the fact that both of them use code to enforce, before or after the generation, a rule on the output. When talking about PICARD and LMQL however, they focus only on the form level: A syntactically valid query or well-formed number where it belongs, but say nothing about the correspondence of the value to the source or the free-text cause belonging to the domain.

2.6 Semantic layers and knowledge-graph

There are two ideas from the data-engineering world that are close to what I need. The first one is the semantic layer, used by tools like data build tool (dbt) and data Cube in which the definition of a metric appears only in a single governed and controlled place [14] [15]. The core principle is a single source of truth: the value is computed in one place and read everywhere else so two reports, for example, cannot disagree on the calculations because they have silently built their own version of the metric. This methodology is the one that I adopt for numbers: take a figure from a single upstream source and copy it, never letting the model work it out. However, that approach is limited because the semantic layer stops at the narrative. It returns clean and canonical numbers, but it doesn't say why that number moved or list, among the possible causes, the one worth reporting.

The second idea regards the causal side. Grounding an LLM in a knowledge graph is the best way to keep what the model says inside a fixed domain. How the two can be joined is, by now, well mapped [16]: a graph defines what the domain treats as true and guides the model's reasoning, preventing it from relying on whatever has been absorbed during the training. What matters the most for my case is the Reasoning over Graphs (RoG). It is an approach that "bounds" every step in the generation process to a specific path in the graph, such that a response can be traced back to a chain of specific relations rather than to a plausible-sounding guess [17]. Other strategies directly feed a smaller version of the tree into the prompt itself [18], but independently of the methodology that is chosen, the key idea is to give the model a structure and let that structure decide what counts as

admissible to say.

Neither family does the whole job alone. The semantic layer return the right number, no story and the knowledge tree grounding bound the story, but nothing about the value. What is needed is a combination of both to make sure that the framework is capable of supporting a correct operational narrative, and it is the one I build in chapter 4.

2.7 Generative BI and conversational analytics

The applied side of turning dashboards into narrative goes under different names: Conversational BI, augmented analytics, generative BI. Its oldest thread is text-to-SQL, where a natural language question is translated into an executable query, scored by benchmark like Spider that evaluates how effective and efficient the transformations is [19]. A parallel line turns the same question into a chart [20]. Two different outputs originated from the same idea: a table or a picture, nothing more. Any explanation, that a manager needs is outside their scope.

It is proven that letting a model handle the numbers is risky. LLM are unreliable even on simple quantitative reasoning, because of their nature as stated in section 2.3. A study about text-to-SQL reveals that, in a production environment, the BI reports present semantic hallucinations, specifically regarding the meaning, with no domain-specific way to track them down[21]. This is the empirical reason behind the strict rule implemented in chapter 4: the safest number is the one the models never compute.

Over time commercial tool have moved closer to the narrative side. Tableau Pulse [22] and Copilot for Power BI [23] are capable of writing short insights over tiles, and they are the industry baseline for this work. The most advanced of them publishes an approach close to the one that I adopt: Tableau Pulse use a deterministic statistical model as ground truth for generation, keeping the LLM away from numbers. None of those tools, however, expose how that grounding holds up under measurement.

Academic field has grown fast, on this topic, during the last year. Some systems

are capable of detecting the domain by their own without any human direction and elaborate multi-perspective narratives without any predefined schema or concepts [24]. Others, especially in finance, join casual-discovery statistical algorithms with budgetary variance to automate root cause diagnosis [25], which is the case nearest to mine.

2.8 Research Gap

Identification of the specific gap in the current literature that this thesis aims to bridge by proposing a unified framework.

I suggest referencing stuff as follows: fig. 2.1 or Figure 2.1

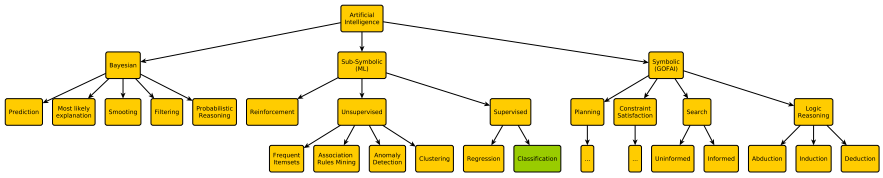

Figure 2.1: Some random image

Chapter 3

Case Study: Context and Problem

3.1 Setting

The last-mile delivery station environment, the continuous improvement cycle (plan, execute, measure, adjust), and the time pressure that makes delayed data visibility a missed intervention window.

3.2 Core Metric

Definition of the reference indicator as a productivity ratio (output units divided by input resource), the gap between planned and actual values, and its additive decomposition into a fixed set of mutually-exclusive categories with a two-level hierarchy, heterogeneous aggregation weights, and a residual term that guarantees completeness.

3.3 Pre-AI Baseline

The manual workflow for monitoring (weekly cadence, tile-by-tile extraction) and for producing the bridge document (subjective cause selection, unconstrained ranking, experience-dependent actions).

3.4 Three Weaknesses

Limited reactivity, subjective root-cause selection, and analyst dependence, mapped explicitly to the three contracts proposed in Chapter 4.

3.5 Methodology

Single-case longitudinal design where the researcher is embedded in the operational team for six months. Development followed an iterative pattern: observe a failure in the AI output, isolate it, fix it, validate against real data, check for regressions. Failure instances were collected and classified into families for the analysis in Chapter 5. A synthetic dataset preserving the structural properties of the real data built for the controlled evaluation.

Chapter 4

Proposed Solution: Three-Contract Framework

4.1 Core Concept

Introduction of the three contracts (tile-as-source, curated knowledge tree, and post-generation sanitizer) as a unified solution to the grounding problem, each acting at a different point of the generation chain.

4.2 Prompt Engineering

The design of the system prompt as an explicit encoding of fidelity rules, with one directive per observed failure mode rather than generic accuracy instructions.

4.3 Architecture

The overall system architecture of the AI-BI pipeline, from data aggregation to final narrative output.

```
public class HelloWorld {
  public static void main(String[] args) {
    // Prints "Hello, World" to the terminal window.
    System.out.println("Hello, World");
  }
}
```

4.4 Application

How the framework was derived from the iterative development of the case study, showing the concrete implementation of each contract.

You may also put some code snippet (which is NOT float by default), eg: section 4.4.

4.5 Fancy formulas here

Chapter 5

Results and Evaluation

5.1 Failure Analysis

A catalog of the main grounding failure modes observed and an explanation of how the proposed framework mitigates them.

5.2 Quantitative Assessment

A controlled evaluation using mock data to compare different system configurations.

5.3 Qualitative Assessment

Two to three real-world qualitative episodes drawn from the case study to provide practical, operational evidence of success.

Chapter 6

Discussion and Future Directions

6.1 Synthesis

How the observed results successfully answer the initial research question.

6.2 Contributions

The specific value added to the existing literature regarding LLMs and BI.

6.3 Generalizability

An assessment of how well the three-contract framework can be adapted and applied to other contexts or industries.

6.4 Limitations

A transparent look at the limits of the current thesis work.

6.5 Future Developments

Potential next steps, such as extending the framework to support interactive Q&A functionalities based on the same underlying schema.

Bibliography

[1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*. Curran Associates, Inc., 2020.

[2] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Yejin Bang, Delong Chen, Wenliang Dai, Ho Shu Chan, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023.

[3] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems*, 43(2):1–55, 2025.

[4] Ankur P. Parikh, Xuezhi Wang, Sebastian Gehrmann, Manaal Faruqui, Bhuwan Dhingra, Diyi Yang, and Dipanjan Das. Totto: A controlled table-to-text generation dataset. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1173–1186. Association for Computational Linguistics, 2020.

[5] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. Ragas: Automated evaluation of retrieval augmented generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for*

Computational Linguistics (EACL): System Demonstrations, pages 150–158. Association for Computational Linguistics, 2024.

[6] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, pages 24824–24837. Curran Associates, Inc., 2022.

[7] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*. OpenReview.net, 2023.

[8] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, pages 27730–27744. Curran Associates, Inc., 2022.

[9] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*. OpenReview.net, 2023.

[10] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*. Curran Associates, Inc., 2023.

[11] Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. Pal: Program-aided language models.

In *Proceedings of the 40th International Conference on Machine Learning (ICML)*. PMLR, 2023.

[12] Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. Prompting is programming: A query language for large language models. *Proceedings of the ACM on Programming Languages*, 7(PLDI):1946–1969, 2023.

[13] Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. Picard: Parsing incrementally for constrained auto-regressive decoding from language models. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 9895–9901. Association for Computational Linguistics, 2021.

[14] dbt Labs. dbt semantic layer and metricflow. `https://docs.getdbt.com/docs/build/about-metricflow`, 2024.

[15] Cube Dev. Cube: the semantic layer for data apps. `https://cube.dev/`, 2024.

[16] Shirui Pan, Linhao Luo, Yufei Wang, Chen Chen, Jiapu Wang, and Xindong Wu. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering*, 36(7):3580–3599, 2024.

[17] Linhao Luo, Yuan-Fang Li, Gholamreza Haffari, and Shirui Pan. Reasoning on graphs: Faithful and interpretable large language model reasoning. In *International Conference on Learning Representations (ICLR)*. OpenReview.net, 2024.

[18] Jinheon Baek, Alham Fikri Aji, and Amir Saffari. Knowledge-augmented language model prompting for zero-shot knowledge graph question answering. In *Proceedings of the First Workshop on Matching From Unstructured and Structured Data (MATCHING 2023)*, pages 70–98. Association for Computational Linguistics, 2023.

[19] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and

Dragomir Radev. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3911–3921. Association for Computational Linguistics, 2018.

[20] Leixian Shen, Enya Shen, Yuyu Luo, Xiaocong Yang, Xuming Hu, Xiongshuai Zhang, Zhiwei Tai, and Jianmin Wang. Towards natural language interfaces for data visualization: A survey. *IEEE Transactions on Visualization and Computer Graphics*, 29(6):3121–3144, 2023.

[21] Jeho Choi. Fact-consistency evaluation of text-to-sql generation for business intelligence using exaone 3.5. *arXiv preprint arXiv:2505.00060*, 2025.

[22] Salesforce Tableau. Tableau pulse: Ai-generated metric insights. `https://www.tableau.com/products/tableau-pulse`, 2024.

[23] Microsoft. Copilot in power bi. `https://learn.microsoft.com/power-bi/create-reports/copilot-introduction`, 2024.

[24] Ran Zhang, Mohannad Elhamod, et al. Data-to-dashboard: Multi-agent llm framework for insightful visualization in enterprise analytics. *arXiv preprint arXiv:2505.23695*, 2025.

[25] Hejing Chen, Yixuan Lu, Yijing Wei, Jinyan Lyu, Ruibo Wu, and Chen Chen. Causal-llm: A hybrid framework for automated budgetary variance diagnosis and reasoning. In *Proceedings of the 2026 International Conference on Big Data and Internet of Things and Cloud Networks (BDICN)*. Association for Computing Machinery, 2026.

Acknowledgements

Optional. Max 1 page.